心音边缘 AI 学习课：L18-L36 离线读物

用途：外出期间不带电脑，也能继续把课程读完。
生成日期：2026-10-09
配套仓库：C:\Users\psr\Documents\ChatGPT\FYP

怎么用这份文档

每一课都按同一个结构写：

1. **这一课要回答的问题** —— 一句话。
2. **原理** —— 为什么是这样，用生活里的例子讲。
3. **本仓库的真实数字** —— 全部来自仓库里已有的报告，不是我编的。
4. **回来后要做什么** —— 能动手时补上的操作。
5. **思考题** —— 想完再看最后的答案部分。

两条重要说明：

- L18 到 L24 的数字来自仓库里已经跑完的报告，现在就能读完。
- L25 到 L36 大部分需要在开发板上实测。这份文档给你**原理和清单**，但凡是没测过的，一律标"**待实测**"，不假装有数据。这正是 ADR 规则要求的：证据不足就标候选。

建议节奏：每天 1 到 2 课，读完先合上文档回忆一遍，再看思考题。

第一部分：模型与量化（L18-L24）

这一段的核心问题是：**我们手上的模型到底有多好、有多大、多快，以及它缩小以后还准不准。**

L18 跑 train_cnn.py，读报告

这一课要回答的问题

CNN 是怎么被训练出来的？报告里哪些数字能信，哪些不能信？

原理

训练就是四步循环，重复很多遍：

1. **前向**：把一批窗口喂进模型，得到预测。
2. **算损失**：预测和正确答案差多少。
3. **反向**：算出"每个权重该往哪个方向改"。
4. **更新**：挪一小步。

重复 40 轮（`--epochs 40`），每轮结束时在**验证集**上打一次分，分数最好的那一轮存档。连续 8 轮没有进步就提前停（`--patience 8`）。这个机制叫**早停**。

> 类比：每做完一章练习就做一次模拟考。模拟考连考 8 次都没进步，就别硬刷了，把最好的那次成绩交上去。

本仓库的真实数字

小数据集的 CNN 报告（24 名受试者）：

```
稳定（由数据和模型结构决定，重跑不会变）：
  窗口数：train 216 / val 44 / test 56
  参数量：41,105
  FP32 权重：162.0 KiB
  INT8 权重估算：40.1 KiB
  单窗口 MACs：5,722,944

基本稳定（由训练决定）：
  最佳轮次：第 2 轮
  窗口级 test AUROC：0.694

会变的（测试患者只有 4 人，换台机器就不一样）：
  验证集选出的阈值：0.46 到 0.49 之间
  患者级 test 敏感度/特异度：两次运行分别得到 (1.000, 0.000) 和 (1.000, 1.000)
  PC 上 PyTorch 延迟：约 0.8 到 1.5 ms（随机器和负载变化）
```

两个必须看懂的数字：

第一，"最佳轮次是第 2 轮"。 一个 41,000 参数的模型，训练两轮就停了，等于刚翻开书就交卷。这说明数据集太小，验证集分数一开始就抖动，早停被"噪声"骗了。

第二，"只有 4 个测试患者"。 同一份代码、同一个种子，我在两台机器上跑出过这样的结果：

```
第一次：患者级 test 准确率 0.500，敏感度 1.000，特异度 0.000   <- 全部判"有病"
第二次：患者级 test 准确率 1.000，敏感度 1.000，特异度 1.000   <- 全部判对
```

窗口级 AUROC 两次都是 0.694（稳定），但划定阈值之后，患者级结论可以从"全错"跳到"全对"。**这不是 bug，是样本量太小。**

所以：小数据集上的漂亮数字，先怀疑，再相信。

一个真的踩过的坑：缓存陷阱

```
windows-train.npy / windows-val.npy / windows-test.npy
这些文件名里不含预处理参数（窗口长度、步长、每条录音最多几窗）。

train_cnn.py 发现缓存存在就直接用，不会检查缓存是不是用同一组参数生成的。
```

我第一次重跑时就中招了：命令里用的是默认参数（每条录音最多 4 窗，应该得到 216 个训练窗口），但目录里的缓存是更早用"不限制窗口数"生成的。报告照常输出，窗口数却变成了 **711**，一切看起来都很正常。

两条教训：

1. 改过窗口参数之后，命令里必须加 `--rebuild-cache`。
2. 看报告的第一件事是**核对窗口数**，不是看准确率。

回来后要做什么

```
.\.venv\Scripts\python.exe tools\ai\train_cnn.py --data-dir data\processed\circor-subset --out-dir
data\reports\circor-subset-cnn --rebuild-cache
```

跑完对着 `report.md` 核对四件事：窗口数是不是 216/44/56、最佳轮次、窗口级 AUROC、患者级敏感度/特异度。

思考题

1. 为什么"最佳轮次 = 2"是个危险信号？

2. 同一份代码、同一个种子，两次的患者级特异度一次是 0.000、一次是 1.000，这说明什么？
3. 如果改了 `--hop-sec` 却忘了加 `--rebuild-cache`，报告会出什么问题？

L19 窗口筛选：all 和 annotated 哪个好

这一课要回答的问题

只用"有标注的高质量片段"来训练，模型会变好吗？

原理

回想 L13：CirCor 只标注高质量片段，其他是 0。所以有两种做法：

- **all**：所有窗口都用。样本多，但包含静音段、噪声段。
- **annotated**：只用标注比例 ≥ 0.5 的窗口。样本干净，但数量变少。

> 类比：复习时用"全部笔记"还是"老师划过重点的笔记"？
> 全部笔记信息更全，但里面有你抄错的、无关的；重点笔记干净，但可能漏东西。

另外要记住一件事：**窗口的标签是从患者标签复制来的**，这叫**弱标签**。一个患者"有杂音"，他所有窗口都标 1，但杂音其实只在收缩期那一段。所以 all 里面必然混着一批"标签说有病、实际没杂音"的窗口。

本仓库的真实数字

完整数据集 5 折患者级 AUROC：

模型	窗口策略	窗口数	患者级 AUROC	患者级 BA	敏感度	特异度
逻辑回归	all	12028	0.775 ± 0.029	0.701	0.637	0.765
逻辑回归	annotated	11728	0.789 ± 0.031	0.703	0.482	0.924
1D CNN	all	12028	0.811 ± 0.052	0.730	0.525	0.934
1D CNN	annotated	11728	0.834 ± 0.061	0.718	0.626	0.810

三个关键观察：

1. **方向一致**：两种模型上，annotated 的 AUROC 都更高（+0.014 和 +0.023）。
2. **窗口数几乎没变**：12028 → 11728，只少了 2.5%。也就是说完整数据里绝大多数窗口本来就在标注区内。
3. **敏感度和特异度在换位置**：CNN 的敏感度从 0.525 升到 0.626，特异度从 0.934 降到 0.810。这不是"变好"，是**工作点移动了。**

所以要诚实地说：annotated 方向支持，但提升幅度不大（+0.02 左右），而且标准差更大（0.061）。它只算"候选策略"，不算定论。

> 还有一个反差值得记住：在 24 人的小数据集上，annotated 让窗口数减少了 45%~46%；而在 874 人的完整数据上只减少 2.5%。同一件事在不同数据集上效果差这么多，说明小数据集的比例会骗人。

回来后要做什么

```
powershell -NoProfile -File tools\ai\run_window_selection_experiments.ps1
```

跑完把四份报告里的 AUROC 和窗口数抄进一张表，再决定 annotated 要不要升级为正式策略。

思考题

1. 为什么"窗口少了但模型更好"是有可能的?
2. 敏感度升、特异度降，为什么不能简单说"模型变好了"?

L20 类别不平衡和 pos_weight

这一课要回答的问题

数据一边倒的时候，怎么防止模型偷懒?

原理

先看我们数据的真实比例:

```
有标签的受试者：874 人
  健康（Absent）：695 人   = 79.5%
  异常（Present）：179 人   = 20.5%
比例：约 3.9 : 1
```

如果模型一律猜"健康"，准确率就有 **79.5%**。看起来还不错，其实一个病人都没找出来。

这就是准确率的陷阱。 数据一边倒时，准确率会给你虚假的安心。

解决办法是加权:

```
pos_weight = 健康人数 / 病人数 = 695 / 179 ≈ 3.88
```

意思是:**答错一个病人，惩罚乘以 3.88。**

> 类比:老师说"这次考试，做错基础题扣 1 分，做错压轴题扣 4 分"。它会逼你把注意力放在难的那部分。

仓库里两条基线都做了这件事，只是名字不同:

- 逻辑回归:`balanced=True`，给少数类更高的样本权重
- CNN:`BCEWithLogitsLoss(pos_weight=...)`

本仓库的真实数字

```
874 名受试者：695 健康 + 179 异常
比例 3.9 : 1
每折测试约 175 人（约 139 健康 + 36 异常）
```

注意一个细节:加权并不能"变出"信息。它只是让模型别忽略少数类。副作用是**假阳性会变多**（宁可错报也不漏报）。

这在医学筛查里通常是可接受的取舍，但要清楚代价。ADR-007 里那句"CNN annotated 特异度较低（0.810 vs 0.934）"就是这个副作用的表现。

回来后要做什么

用同一份数据、同一个模型，只把 pos_weight 从 3.88 改成 1.0，对比四件事:AUROC、敏感度、特异度、混淆矩阵。

思考题

1. 874 人里 695 健康、179 异常。如果模型一律猜"健康"，准确率是多少?为什么这个数字没有意义?
2. 加权之后假阳性变多，为什么医学筛查还愿意这么做?

L21 阈值选择和校准

这一课要回答的问题

概率多高才算"有病"？为什么不能直接定 0.5？

原理

模型输出的是一个概率，比如 0.43。判断成"有病"需要一条线，这条线就是**阈值。**

阈值不能拍脑袋，也不能看答案定。 正确做法是在**验证集**上试各种阈值，挑表现最好的那个。仓库里的 `best_threshold` 函数就是干这个的，它挑的是"平衡准确率最高"的阈值。

> 类比：及格线定在哪里，要拿模拟考来试，不能用期末考来试。用期末考定线，等于提前偷看了答案。

还有一个更高级的概念叫校准。 校准好意味着：模型说"70% 有病的那些人"里，真的有大约 70% 有病。

如果校准不好，会出现这种情况：同一个模型，在 A 折最佳阈值是 0.15，在 B 折是 0.71。**阈值到处乱跑，说明概率本身不可信。**

本仓库的真实数字

```
小数据集逻辑回归：验证集选出的阈值 = 0.4213
小数据集 CNN     ：验证集选出的阈值 = 0.4614
完整数据 CNN annotated：5 折的阈值跨度 = 0.15 到 0.71
```

那个 **0.15 到 0.71**
是这份文档里最重要的一个数字之一。它说明：**目前模型的概率还没有校准好，所以不能冻结任何单一阈值。**

ADR-007 里"不冻结单一阈值"这条结论，就是从这来的。

另一个容易踩的坑：pooled 指标和跨折均值会不一样。

```
CNN annotated：跨折均值 0.834，pooled 0.802
```

原因就是各折阈值不同、概率标定不同。合在一起算，效果会掉下来。所以跨折均值用来判断**判别能力**，而 pooled 指标用来检查**整体校准。**

回来后要做什么

把 5 折的阈值画成一条线（0.15 到 0.71），再找出每折的最佳阈值和对应的 BA。看阈值和 BA 之间有没有明显关系。

思考题

1. 为什么阈值只能在验证集上选？
2. 5 折的阈值从 0.15 跳到 0.71，说明模型的哪方面有问题？

L22 参数量、MACs 和 PC 延迟

这一课要回答的问题

怎么数清一个模型"有多大、多快"？

原理

这里有三个不同的东西，很多人会混：

名字	回答的问题	单位
参数量	要记多少个数字	个
MACs	一次推理要做多少次乘加	次
延迟	一次推理要花多少时间	毫秒

类比：

- 参数量 = 一本书有多少页
- MACs = 读一遍要翻多少次
- 延迟 = 你读一遍要多久

页数少的书，不一定读得快（可能字很密）；读得快的人，不一定书薄（可能他熟练）。

为什么用 P95 而不是平均值?

平均 0.75 毫秒听起来很好，但如果 5% 的情况下要 20 毫秒，用户体验就是"偶尔卡一下"。P95 的意思是"95% 的情况都不超过这个数"，它更接近用户的真实感受。

最重要的一条纪律：PC 延迟不能外推到 MCU。

PC 是几个 GHz、带缓存和多核；ESP32-S3 是 240 MHz 的单片机。差距不是几倍，是几十到几百倍。**必须上板实测。**

本仓库的真实数字

```
参数量                ：41,105
单窗口 MACs           ：5,722,944
FP32 权重             ：162.0 KiB
INT8 权重估算         ：40.1 KiB
PC PyTorch 延迟       ：平均 0.75 ~ 1.04 ms（两台机器各测一次，会变）
PC INT8 ONNX 延迟     ：平均 0.12 ms，P95 0.14 ms
每秒需要的窗口数      ：0.5（每 2 秒一个窗口）
平均 MACs/s           ：5,722,944 × 0.5 = 2,861,472
```

由这些数字能推出什么?

- 0.5 窗/秒，说明实时性要求其实很宽松：**整整 2 秒**要处理一个窗口。
- 那么瓶颈根本不在算力，而在**内存和并发**（BLE 收数据、显示、Wi-Fi 同时跑）。
- 这是一个重要的工程判断：不要一上来就担心算力不够。

回来后要做什么

```
.\.venv\Scripts\python.exe tools\ai\train_cnn.py --data-dir data\processed\circor-subset --out-dir
data\reports\circor-subset-cnn
```

在 `report.md` 的 "Budget snapshot" 一节里核对上面每一个数字。

思考题

1. 参数量少，为什么不代表跑得快?
2. 为什么报告里看重 P95 而不是平均值?

L23 ONNX 导出和 FP32/INT8 量化

这一课要回答的问题

训练好的模型，怎么变成单片机能读懂的文件?

原理

第一步：换成通用格式。

PyTorch 的模型文件只有 PyTorch 自己能读。所以先导出成
ONNX，这是一种"大家都认识的中间格式"，之后可以再转成 TFLite、ESP-DL 等其他格式。

> 类比：ONNX 像 PDF。不管原文件是 Word 还是 Pages，转成 PDF 之后谁都能看。

第二步：量化。

训练出来的权重是 **FP32**（每个数 4 字节）。量化就是把它换成 **INT8**（每个数 1
字节），体积直接小到四分之一。

> 类比：本来记体温写 36.53℃，现在只记刻度值 36。省地方，但精度有损失。

难点在于：不能随便砍。

必须先拿一批真实数据跑一遍，看每层的数值范围，再决定"刻度尺"怎么画。这一步叫**校准**。没有校准的量化，模型可能直接废掉。

仓库里的 `quantize_onnx_static` 用的就是静态量化 + 逐通道（per-channel）方案，需要提供校准窗口。

本仓库的真实数字

```
FP32 权重        : 162.0 KiB
INT8 权重估算    : 40.1 KiB      <- 少了 4 倍
INT8 ONNX 文件   : 68.8 KiB      <- 比 40.1 大，正常
PC INT8 延迟     : 平均 0.12 ms（FP32 是 0.75 ~ 1.04 ms）
```

为什么文件 68.8 KiB，而权重估算只有 40.1 KiB?

因为文件里除了压缩后的权重，还要存:

- 每一层的**刻度表**（scale 和 zero-point）
- 一些没法量化的浮点部分
- 文件头和结构信息

所以"权重估算"和"文件大小"是两个不同的数字，上板算 Flash 时要看文件大小。

还有一个隐形好处：很多单片机做整数乘法比浮点乘法快，所以量化通常同时省空间和时间（这里就是 0.75 → 0.12 毫秒）。

回来后要做什么

```
.\.venv\Scripts\python.exe tools\ai\train_cnn.py --data-dir data\processed\circor-subset --out-dir
data\reports\circor-subset-cnn
```

在输出目录里找到 `model-fp32.onnx` 和 `model-int8.onnx`，比较两个文件的实际大小。

思考题

1. 为什么 INT8 文件（68.8 KiB）比 INT8 权重估算（40.1 KiB）大?
2. 量化之前为什么必须做校准?

L24 量化前后的指标对比和失败判据

这一课要回答的问题

量化之后模型还准吗？准到什么程度才算"可以接受"？

原理

量化一定会有精度损失，问题只是损失多少。所以必须做一件事：

用同一个测试集，分别跑 FP32 和 INT8，比较患者级 AUROC。

判据是提前定好的，不是事后补的。 ADR-007 规定了三条：

```
患者级 AUROC          ≥ 0.70
患者级 BA             ≥ 0.65
INT8 的 AUROC 下降    ≤ 0.03
```

提前定判据的意义在于：**防止你看到结果之后，再把标准改成"刚好通过"。** 这是工程诚信的一部分。

本仓库的真实数字

```
FP32 患者级 AUROC（完整数据 5 折）：
  逻辑回归 all 0.775 / annotated 0.789
  1D CNN   all 0.811 / annotated 0.834

INT8 量化后的指标：待实测
```

"待实测"不是偷懒。 量化必须在目标推理框架上跑真实数据才有意义，PC
上估算的数字不能当验收依据。这正是 ADR 规则里"证据不足就标候选"的要求。

回来后要做什么

1. 用同一个测试集，分别跑 FP32 ONNX 和 INT8 ONNX，得到两套患者级分数。
2. 算两个 AUROC，做差。
3. 对照三条判据，写明"通过"或"不通过"。
4. 如果不通过，回退路径是：先做**量化敏感度分析**找出哪一层最敏感，再考虑量化感知训练（QAT），或者干脆保留 FP32 并接受更大的 Flash 占用。

思考题

1. 为什么量化对比必须用同一个测试集？
2. 如果 INT8 的 AUROC 下降了 0.05，第一步该查什么？

第二部分：从 PC 到开发板（L25-L30）

这一段的核心问题是：**模型怎么真的跑到一块单片机上，并且跑得动、跑得快。**

重要提醒：这一部分开始，几乎所有数字都需要实物。文档给你原理和检查清单，但没有实测的地方一律标"待实测"。

L25 TFLite Micro 和 ESP-DL 基础

这一课要回答的问题

单片机是怎么"跑"一个神经网络的？

原理

单片机上没有操作系统，也没有 Python。所以要一个**推理框架**，它包含两部分：

- **解释器**：读模型文件，按顺序执行每一步。
- **算子内核**：卷积、池化、全连接这些操作的 C 语言实现。

最关键的区别是内存。 PC 上内存随便用，单片机上内存是固定的，而且**不能动态申请。**

所以推理框架要求你提前划一块固定的内存（叫 **tensor arena**），所有中间结果都在这块内存里反复复用。

> 类比：PC 上做饭可以随时去超市买材料；单片机上做饭，必须先把所有可能要用的材料一次性摆到台面上，台面多大就只能做多大的菜。

ESP32-S3 的特别之处：它有向量指令（SIMD），做 INT8
运算效率比纯软件模拟高很多。所以在这块板上，**INT8 几乎是必选项。**

本仓库的真实数字

```
模型：41,105 参数，INT8 ONNX 68.8 KiB
目标板 tensor arena 大小：待实测
目标板峰值 RAM：待实测
```

回来后要做什么

1. 确认 ESP-DL 或 TFLite Micro 是否支持我们模型用到的全部算子（Conv1d、BatchNorm、ReLU、MaxPool、GlobalAveragePool、Linear）。
2. 记录框架要求的 arena 大小。

思考题

1. 为什么单片机上的推理框架要求"提前划一块固定内存"？
2. 为什么 ESP32-S3 上 INT8 比 FP32 更有优势？

L26 把模型转成目标板格式

这一课要回答的问题

从 ONNX 转成目标板格式，最容易在哪里出错？

原理

转换过程中最容易出问题的是**算子不兼容**：ONNX 里有一个算子，目标框架没实现，或者实现方式不一样。

> 类比：你把一份 Word 文档转成 PDF，大部分排版能保住，但某些特殊的艺术字可能变成方块。

所以转换完必须做"数值对齐"检查：

同一个输入，分别喂给 PC 上的 ONNX 和目标板上的模型，比较两边输出的差异。差异必须小到可以忽略。

这一步不能跳过。 如果跳过，你可能在开发板上看到一个"能跑但结果完全不对"的模型，然后花几天去查一个本来五分钟就能发现的问题。

本仓库的真实数字

```
转换后的算子兼容性清单：待实测
PC 与目标板的输出差异：待实测
```

回来后要做什么

1. 用同一段 2 秒心音，分别在 PC 的 ONNX 和开发板上推理。
2. 打印两边输出的原始分数，算差值。
3. 差值应该非常小（同一套权重、同样的量化，差异只应来自计算顺序和舍入）。
4. 如果差异大，按层排查：从第一层开始，逐层比较输出。

思考题

1. 为什么转换完必须做数值对齐，而不能只看"能不能跑起来"？
2. 如果发现第 3 层之后差异突然变大，最可能是什么原因？

L27 安装 ESP-IDF，跑最小示例

这一课要回答的问题

拿到开发板，第一件事做什么？

原理

先跑通最小示例，再碰我们的模型。

这不是浪费时间。因为一旦后面出问题，你需要知道"是环境问题"还是"我的代码问题"。先跑通官方示例，就等于先把"环境问题"这个嫌疑排除了。

> 类比：装修前先验收水电。水管本身有问题，跟你买了什么家具无关。

同时要记录**基线数字**：没跑任何模型时，这块板还剩多少可用内存。后面测峰值 RAM 时，要跟这个基线对比。

本仓库的真实数字

```
ESP32-S3：双核 LX7 240 MHz、512 KB 内部 SRAM、可外接 PSRAM
启动日志和 heap 基线：待实测
```

回来后要做什么

1. 安装 ESP-IDF，编译并烧写 hello world 示例。
2. 从启动日志里抄下：芯片型号、Flash 大小、PSRAM 大小。
3. 在程序里打印空闲 heap，记下这个基线数字。

思考题

1. 为什么上板第一件事是跑官方示例，而不是直接烧我们的模型？
2. heap 基线有什么用？

L28 在 ESP32-S3 上跑第一次推理

这一课要回答的问题

第一次推理成功，要检查哪几件事？

原理

"跑出了一个数"不等于"跑对了"。 第一次推理要同时看三件事：

1. **结果合不合理**：输出分数在合理范围内，且和 PC 上对同一段输入的结果接近。
2. **花了多久**：单次推理的执行时间。
3. **内存用了多少**：推理前后的 heap 变化。

   类比：新买的车第一次上路，你要同时看：它走的方向对不对、速度多少、油耗多少。只看"它动了"是不够的。

本仓库的真实数字

```
PC 上的参考值：INT8 ONNX 平均 0.12 ms
目标板第一次推理的输出、时间、内存：待实测
```

回来后要做什么

1. 用一段固定输入（可以把输入全设为 0，或者用一段真实心音）。
2. 在 PC 和目标板各跑一次，比较输出。
3. 记录耗时和 heap 变化。

思考题

1. 为什么第一次推理要同时检查结果、时间、内存三项?
2. 如果开发板输出和 PC 差很多，但代码看起来没错，你从哪里开始查?

L29 测量峰值 RAM 和 P95 时延

这一课要回答的问题

为什么平均时延会骗人？峰值内存怎么测才准?

原理

关于时延：平均值会把偶发的慢点藏起来。所以要记录三档：

```
平均 / P95 / 最大
```

P95 = 95% 的情况都不超过这个值。用户感受到的"卡"，基本都发生在 P95 以外那 5%。

关于内存：峰值 RAM **不等于**模型文件大小。

一个 68.8 KiB 的模型，运行时可能要占几百 KiB，因为还包括：

```
权重
输入张量
每层的中间结果（这一项通常最大）
算子工作区
框架本身的开销
对齐余量
```

测量方法：在推理前记录一次 heap，推理中记录一次，推理后再记录一次，取最低点。

本仓库的真实数字

预算门槛（来自资源预算指南，都是候选值，必须用真板验证）：

```
模型文件    < 可用 Flash/PSRAM 的 50%
峰值 RAM    < 可用内存的 70%
推理时延    P95 < 允许显示更新周期的 50%
```

```
目标板峰值 RAM：待实测
目标板 P95 时延：待实测
```

回来后要做什么

1. 跑 1000 次推理，记录平均、P95、最大。
2. 用 heap 探针记录推理过程中的最低点。
3. 对照上面三条门槛写结论。**没达标就保持"候选"，不要写成"已冻结"。**

思考题

1. 为什么"模型文件 68.8 KiB"不能推出"峰值 RAM 只要 68.8 KiB"？
2. 如果平均 3 ms 但最大 40 ms，你能接受吗？要看什么条件？

L30 测量电流和功耗

这一课要回答的问题

这块板能靠电池跑多久？

原理

平均电流和峰值电流是两个不同的数字，用途也不同：

```
峰值电流 -> 决定电池能不能供得上（不够会重启或掉电）
平均电流 -> 决定能用多久（续航）
```

续航估算的基本公式：

```
续航时间(小时) ≈ 电池容量(mAh) / 平均电流(mA)
```

但要注意：这个公式只在电流稳定时成立。如果设备是"大部分时间睡觉、偶尔醒来推理"，就要按**占空比**加权算。

> 类比：算一辆车加满油能跑多远，不能只看它最高速度时的油耗，要看它在市区和高速各占多少时间。

本仓库的真实数字

```
目标板平均电流：待实测
目标板峰值电流：待实测
连续推理续航：待实测
```

回来后要做什么

1. 分别测：待机时、只有 BLE 时、推理时、Wi-Fi 上传时的平均电流。
2. 测一次峰值电流。
3. 按实际使用场景（推理频率、Wi-Fi 上传频率）估算续航。

思考题

1. 为什么峰值电流和平均电流要分开测？
2. 如果设备 90% 时间在待机，为什么要按占空比算续航？

第三部分：并发与验收（L31-L36）

这一段的核心问题是：**所有东西同时跑的时候，系统还稳不稳。**

L31 BLE 接收和推理同时运行

这一课要回答的问题

推理的时候，音频还在进来吗？

原理

这是整个项目里最容易出事的地方。

推理要占 CPU 和内存，BLE 收数据也要占 CPU 和内存。如果推理把 CPU 占满了，BLE 就会来不及处理，**音频数据直接丢掉。**

而丢帧的后果是：模型拿到的是一段不连续的声音，判断结果自然不可信。更糟的是，这种错误不会报错，只会静默地让结果变差。

> 类比：餐厅只有一个服务员。他忙着在后厨算账（推理），前台来了客人（BLE 数据）就没人接。客人不会大喊，只会默默走掉（数据丢掉）。

所以必须检查丢帧率，而不是只看"推理成功了没有"。

本仓库的真实数字

```
BLE 连续传输丢帧率：待实测
推理期间的接收延迟：待实测
```

回来后要做什么

1. 让 BLE 持续传音频，同时持续推理。
2. 在接收端统计：期望收到多少个包，实际收到多少个。
3. 丢帧率目标：**推理期间不能有持续性丢帧**（偶发个位数可以接受，需要记录）。

思考题

1. 为什么丢帧比"推理报错"更难发现、也更危险？
2. 如果发现推理期间稳定丢 5% 的包，可以从哪两个方向改进？

L32 Wi-Fi、显示和推理并发

这一课要回答的问题

ESP32-S3 同时开 Wi-Fi 和 BLE，会互相干扰吗？

原理

会，而且这是我们方案里最需要验证的一条。

原因：ESP32-S3 **只有 2.4 GHz 一套射频**，Wi-Fi 和 BLE
要**分时共用**它。所以两者同时工作时，各自的吞吐和延迟都会受影响。

> 类比：一条单车道。两边的车都要过，只能轮流走。轮流的时候，每边都会变慢。

再加上显示：刷屏要占内存带宽和 CPU，如果刷屏和推理挤在一起，屏幕可能掉帧。

所以这一段要测的不是"能不能跑"，而是"同时跑的时候各项指标掉多少"。

本仓库的真实数字

```
Wi-Fi 与 BLE 共存时的吞吐/延迟：待实测
显示刷新帧率：待实测
并发时的推理时延：待实测
```

回来后要做什么

1. 先单独测：只开 Wi-Fi、只开 BLE、只开显示，各记一组数。
2. 再组合测：Wi-Fi + BLE、Wi-Fi + BLE + 显示 + 推理。
3. 对比两种情况下每项指标的退化幅度。

思考题

1. 为什么 ESP32-S3 上 Wi-Fi 和 BLE 会互相影响？
2. 为什么必须"先单测、再组合测"，而不能直接上组合场景？

L33 5 分钟连续测试和丢帧检查

这一课要回答的问题

跑 10 秒没问题，跑 5 分钟还有问题吗？

原理

短测试看不见的问题，长测试才暴露，主要有三类：

1. **内存泄漏**：每轮少一点，几百轮之后崩掉。
2. **碎片化**：内存总量够，但没有连续的整块可用。
3. **偶发冲突**：射频、显示、推理偶尔撞车，概率很低，但 5 分钟里总会撞上几次。

> 类比：新鞋地板上走两步不打脚，走五公里才知道哪里磨。

为什么是 5 分钟？ 因为 5 分钟约等于 150 个 2 秒窗口。这个数量能覆盖几百次推理、几千个 BLE 包，足以让低频问题浮出来。

本仓库的真实数字

```
5 分钟连续测试：待实测
丢帧数、最大延迟、内存变化：待实测
```

回来后要做什么

记录完整的这几项：

```
测试时长
总窗口数 / 成功推理数
BLE 期望包数 / 实际包数 / 丢帧率
推理延迟：平均 / P95 / 最大
起始 heap / 结束 heap
是否崩过、是否重启过
```

思考题

1. 为什么要在长测试里同时记录"起始 heap"和"结束 heap"？
2. 5 分钟测试大概覆盖多少个推理窗口？为什么这个数量有意义？

L34 用实测证据更新 ADR-006 和 ADR-007

这一课要回答的问题

测完的数据，怎么变成正式的工程结论？

原理

这个项目有一条硬规则：**任何影响接口、器件、协议或算法的结论，都必须走 ADR 流程。**

ADR 的格式是固定的：

```
决策问题 / 候选方案 / 主要优势 / 主要代价 / 验证方法 / 状态
```

状态只有三种：

```
待定 -> 候选 -> 已确认（冻结）
```

关键纪律：

- 证据不足时，只能写"候选"，不能写"已冻结"。
- 新证据推翻旧结论时，**保留旧记录**，新增版本和变更原因，不能删历史。
  类比：科学实验记录本。写错了可以划掉重写，但不能撕掉那一页。因为"为什么当初这么判断"本身也是证据。

本仓库的真实数字

```
ADR-002：20~1000 Hz 带宽、8 kHz 采集、4 kHz AI 输入（候选确认）
ADR-006：主机平台（待定，等模型预算）
ADR-007：第一版端侧心音 AI 基线（候选，已升级为原型基线）
```

ADR-007 目前的状态是：判别能力达标，但**量化后的指标、目标板时延、峰值 RAM、电流全部未测**，所以还不冻结。

回来后要做什么

1. 把 L29、L30、L33 的实测数字填进 `docs/engineering-decision-register.md`。
2. 用实测结果判断 ADR-006 的候选平台能不能满足门槛。
3. 更新时写明：证据等级、测试条件、失败判据、回退方案。

思考题

1. 为什么"新证据推翻旧结论"时要保留旧记录?
2. ADR-006 为什么一直"待定"而不是先随便选一个平台?

L35 准备组会对比页

这一课要回答的问题

组会 PPT 里，为什么必须有"方案对比页"?

原理

项目规范明确规定：**每次组会 PPT 都必须包含方案对比页，不能只展示最终选择。**

原因很实际：只讲结论，别人无法判断你的推理过程。必须给出：

```
决策问题 / 候选方案 / 量化对比 / 风险 / 决策依据 / 验证计划 / 回退方案
```

模板在 `meetings/templates/engineering-decision-slides.md`。

> 类比：面试时只说"我选了这个方案"是过不了的。你必须说清楚有哪几个选择、为什么排除其他几个、什么条件下会改回去。

还有一个细节：候选方案里必须包含"维持现状 / 不引入新组件"这一项。这是规范要求的，因为"什么都不改"永远是一个合法选项。

回来后要做什么

1. 挑一个还没冻结的决策（比如主机平台，ADR-006）。
2. 按模板写出对比页：至少两个候选 + 维持现状。
3. 每项数据都要标证据等级（E1 标准/官方、E2 独立实测、E3 厂商宣称、E4 工程估算）。

思考题

1. 为什么候选方案里必须包含"维持现状"?
2. 如果某个候选方案的数据只有厂商宣传，它能用来冻结决策吗?

L36 总复盘和下一步计划

这一课要回答的问题

整条链走完，我们到底做出了什么，还差什么?

原理

复盘模板是固定的：

```
周次：
本周主题：
完成的可复现产物：
新的实测数字：
证据等级和来源：
新学会的名词：
仍然不懂的两个问题：
失败或异常现象：
下周计划：
```

关键要求：必须写"产物在哪"和"数字来自哪次运行"，不能只写"看完了"。

目前这条链上"已经有的"和"还差的"

环节	状态	证据
数据与标签	已完成	CirCor 1.0.3，874 名有标签受试者
信号处理链	已完成	20~1000 Hz 带通、4 kHz、2 s 窗
两条基线	已完成	逻辑回归 0.775、CNN 0.834（患者级 AUROC）
量化	部分完成	INT8 文件已生成，指标待对比
目标板	未开始	无开发板
并发测试	未开始	依赖目标板

结论：软件侧已经走到"可复现的原型基线"，硬件侧一步都还没走。**整条链的风险已经全部集中到后半段。**

回来后要做什么

1. 按 L27 的顺序，先把 ESP-IDF 环境跑通。
2. 拿到开发板后，优先做 L29（内存和时延），因为它决定平台选型。
3. 每完成一项就更新复盘模板，不要攒到最后补。

思考题

1. 为什么说"风险已经全部集中到后半段"？
2. 如果开发板测出来 P95 时延超过门槛，回退路径有哪几条？

思考题参考答案

L18

1. **最佳轮次 = 2 危险在哪**：说明验证集分数一开始就在抖，早停被噪声骗了。模型还没学到东西就交卷，最终指标自然不可信。
2. **两次特异度 0.000 / 1.000 说明什么**：阈值画在 4 个人身上，多一个人少一个人就能把结论翻过来。样本量太小，患者级指标没有统计意义。
3. **忘了加** `--rebuild-cache`：脚本会直接复用旧缓存，按旧的窗口参数训练。报告看起来正常，但窗口数和内容都对不上，所有指标都不能采信。

L19

1. **窗口少了反而更好是可能的**：因为被去掉的是噪声和静音段，它们是"脏标签"。少而干净，可能比多而脏更有用。
2. **敏感度升、特异度降**：这是工作点在敏感/特异权衡线上滑动，不是单纯变好。要由临床需求决定要哪一头。

L20

1. **一律猜健康 = 79.5% 准确率。**这个数字没意义，因为它一个病人都没找出来。

2. **宁愿假阳性多**：筛查阶段漏掉病人（假阴性）的代价远高于虚惊一场。这是取舍，不是错误。

L21

1. **阈值只能在验证集上选**：用测试集选等于偷看答案，测试分数会虚高，失去"最终考核"的意义。
2. **阈值 0.15 到 0.71 说明概率没校准**：模型输出的概率不能直接当"生病的可能性"用。

L22

1. **参数少不代表快**：快慢取决于运算次数（MACs）和硬件。参数是"要记多少"，MACs 是"要算多少"，两回事。
2. **看 P95**：用户感受到的卡顿发生在最慢的那 5%，平均值会把它们藏起来。

L23

1. **INT8 文件比权重估算大**：文件里还要存每层的刻度表（scale/zero-point）、少量浮点部分和文件结构。
2. **必须校准**：量化要把浮点数映射到整数刻度，不先看真实数据的取值范围，刻度就画不准，模型可能直接失效。

L24

1. **必须同一个测试集**：换了测试集，差异里就混进了"数据不同"这个因素，分不清是量化造成的还是数据造成的。
2. **下降 0.05 先查什么**：先做量化敏感度分析，找出哪一层对量化最敏感（通常是第一层或最后一层）。

L25

1. **固定内存的原因**：单片机上动态申请内存会导致碎片化和不确定的失败。提前划一块 arena，所有中间结果在里面复用，行为才可预测。
2. **ESP32-S3 上 INT8 的优势**：它有向量指令，能一次处理多个 INT8，比浮点运算快很多。

L26

1. **为什么要数值对齐**："能跑"和"跑对"是两件事。算子实现不同会产生细微差异，累积起来可能改变判断结果。
2. **第 3 层后差异变大**：最可能是那个位置的算子不被完整支持，或者量化刻度在那里不合适。

L27

1. **先跑官方示例**：先把环境和工具链的嫌疑排除，后面出错才能确定是自己的代码问题。
2. **heap 基线的用途**：它是"什么都没跑"时的起点。后面测峰值 RAM 要和它比才有意义。

L28

1. **三项一起看**：结果对、速度快、内存够，缺任何一个都不能算成功。
2. **输出差很多从哪查**：从第一层开始逐层比较 PC 和目标板的中间结果，找出差异是从哪一层开始变大的。

L29

1. **文件大小不等于峰值 RAM**：运行时还要装输入、中间结果、工作区。中间结果往往是最大的一块。

2. **平均 3 ms、最大 40 ms**：要看 40 ms
   是否会影响显示更新或音频采集。如果只是偶尔一次且不影响功能，可以接受，但必须记录频率。

L30

1. **峰值和平均分开测**：峰值决定电池供不供得上（供电能力），平均决定能用多久（续航）。
2. **按占空比算**：因为电流是分阶段的，直接用最大电流估算会把续航低估很多。

L31

1. **丢帧更难发现**：丢帧不会报错，只会让模型拿到不连续的信号，结果静默变差。
2. **改进方向**：一是给 BLE 更高的中断/任务优先级；二是降低推理的占用（缩短推理时间或分片处理）。

L32

1. **互相影响的原因**：ESP32-S3 只有一套 2.4 GHz 射频，Wi-Fi 和 BLE 必须分时共用。
2. **先单测再组合**：只有先知道单项基线，才能算出组合时的退化幅度。否则你不知道变差是"本来就差"还是"被挤掉了"。

L33

1. **起始和结束 heap**：两者对比才能发现内存泄漏。只看结束时的绝对值，看不出是否在持续减少。
2. **5 分钟约 150 个窗口**：这个量级能覆盖几千次 BLE 包和几百次推理，足以让低频的偶发问题浮现。

L34

1. **保留旧记录**：因为"当初为什么这么判断"也是证据，能防止同样的错误再犯，也能解释结论的演化过程。
2. **ADR-006 待定**：因为平台选型必须等模型预算。先买芯片再定模型，是本项目明确列出的常见错误。

L35

1. **必须包含维持现状**：因为"什么都不改"永远是一个合法选项，忽略它会让对比不完整。
2. **只有厂商宣传的数据不能冻结决策**：厂商宣称属于 E3
   等级，只能用来筛选候选，必须有目标硬件上的实测（E1/E2）才能冻结。

L36

1. **风险集中到后半段**：软件侧已经有了可复现的基线，剩下的未知（内存、时延、功耗、并发）全都要靠实物才能回答。
2. **P95**
   超门槛的回退路径：一是降低模型规模或层数；二是降低采样率或窗长；三是换更强的平台（ADR-006 候选 B/C/D）；四是降低窗口处理频率，接受更慢的显示更新。

回来后的第一件事

按这个顺序做，不要跳：

1. **L18-L24 复现一遍**：跑 `train_cnn.py`，核对报告里的每一个数字。这一步不需要开发板。
2. **L27 环境**：装 ESP-IDF，跑通 hello world。

3. **拿到开发板后先做 L29**：内存和时延决定平台选型，优先级最高。
4. **再做 L31 和 L33**：并发和长测试，这两项决定产品能不能用。
5. **每完成一项就更新** `docs/engineering-decision-register.md`，标清证据等级和待实测项。

记住项目的核心纪律：**没有实测数据的结论，只能写"候选"。**